

Giải Thích Test RabbitMQ Retry Và Dead Letter Queue

Phạm vi: Bài test client dlq trong docker-compose.full.yml

Mục tiêu: Hiểu rõ một poison message được tạo ra như thế nào, đi qua những exchange và queue nào, được retry bao nhiêu lần, và cuối cùng vì sao nằm trong orders.dlq.

1. Ý Tưởng Của Bài Test

Bài test cố tình tạo một message mà consumer không thể xử lý thành công. Loại message này thường được gọi là **poison message**.

Trong lab, poison message không phải message bị sai JSON. Nó vẫn là JSON hợp lệ, nhưng có trường:

```
simulate_error = true
```

Consumer được lập trình để luôn phát sinh lỗi khi nhìn thấy trường này. Vì vậy, dù thử lại bao nhiêu lần, message vẫn thất bại. Điều này giúp kiểm tra cơ chế retry và DLQ một cách có chủ đích.

Luồng tổng quát:



2. Các Thành Phần Cần Phân Biệt

2.1 Exchange

Exchange là nơi nhận message từ publisher và định tuyến message. Exchange không phải nơi giữ backlog lâu dài.

Lab có ba exchange kiểu direct:

Exchange	Vai trò
orders.main	Định tuyến message cần consumer xử lý

Exchange	Vai trò
orders.retry	Định tuyến message cần chờ trước khi thử lại
orders.dlx	Định tuyến message đã thất bại vĩnh viễn

Vì đây là direct exchange, routing key phải khớp với binding key.

2.2 Queue

Queue là nơi giữ message cho đến khi message được consumer nhận và ACK, hoặc được RabbitMQ dead-letter.

Queue	Vai trò
orders.main.q	Chứa công việc consumer cần xử lý
orders.retry.q	Giữ message thất bại trong 3 giây
orders.dlq	Giữ message đã hết lượt retry để điều tra

2.3 Binding Và Routing Key

Binding nối exchange với queue:

```
orders.main -- routing key "order" --> orders.main.q
orders.retry -- routing key "retry" --> orders.retry.q
orders.dlx -- routing key "dead" --> orders.dlq
```

Publisher gửi message vào exchange kèm routing key. RabbitMQ nhìn binding để quyết định queue nhận message.

3. Lệnh Test Thực Sự Chạy Gì?

Lệnh:

```
docker compose -f docker-compose.full.yml \
--profile tools run --rm client dlq
```

Ý nghĩa từng phần:

- -f docker-compose.full.yml: sử dụng cấu hình full lab.
- --profile tools: bật service client, vì service này thuộc profile tools.
- run --rm client: tạo container client tạm thời, chạy xong thì xóa container.
- dlq: truyền đối số dlq cho chương trình bên trong container.

Dockerfile của client khai báo:

```
ENTRYPOINT ["python", "load_test.py"]
```

Do đó lệnh đầy đủ bên trong container tương đương:

```
python load_test.py dlq
```

Trong client/load_test.py, nhánh dlq chạy:

```
elif args.scenario == "dlq":
    r = post({
```

```

        "client_id": "poison",
        "message": "must reach DLQ",
        "amount": 1,
        "simulate_error": True,
    })

```

Hàm post() gửi HTTP request:

```

requests.post(
    "http://api-server:8000/orders",
    json=payload,
    timeout=10,
)

```

Điểm cần nhớ: **client không publish trực tiếp vào RabbitMQ**. Client gọi API; API mới là publisher của message đầu tiên.

4. API Tạo Và Publish Message Đầu Tiên

Endpoint nhận request là:

```

@app.post("/orders", status_code=202)
def create_order(...):

```

API tạo các ID và message:

```

message = {
    "id": order_id,
    "request_id": request_id,
    "trace_id": trace_id,
    "created_at": now,
    **order.model_dump(),
}

```

Message thực tế có dạng gần giống:

```

{
    "id": "order-uuid",
    "request_id": "request-uuid",
    "trace_id": "trace-uuid",
    "created_at": "2026-06-12T...",
    "client_id": "poison",
    "message": "must reach DLQ",
    "amount": 1,
    "simulate_error": true
}

```

API publish message:

```

ch.confirm_delivery()
ch.basic_publish(
    MAIN_EXCHANGE,
    "order",
    json.dumps(message),
    properties=BasicProperties(
        delivery_mode=2,
        content_type="application/json",
        correlation_id=order_id,
        headers={
            "request_id": request_id,
            "trace_id": trace_id,
            "retry_count": 0,
        },
    ),
    mandatory=True,
)

```

)

Ý nghĩa:

- MAIN_EXCHANGE có giá trị orders.main.
- "order" là routing key.
- retry_count: 0 nghĩa là message chưa từng retry.
- delivery_mode=2 yêu cầu message persistent.
- confirm_delivery() yêu cầu RabbitMQ xác nhận publish.
- mandatory=True giúp phát hiện trường hợp exchange không route được message tới queue.

RabbitMQ sử dụng binding:

```
orders.main -- key "order" --> orders.main.q
```

Vì vậy message đầu tiên được đưa vào orders.main.q.

5. Code Khai Báo Topology RabbitMQ

API và consumer đều gọi `declare_topology(channel)`. Hàm này bảo đảm exchange, queue và binding tồn tại trước khi publish hoặc consume.

5.1 Chọn quorum queue

Trong `docker-compose.full.yml`:

```
RABBITMQ_QUEUE_TYPE: quorum
```

Code chuyển giá trị đó thành queue argument:

```
qargs = {"x-queue-type": "quorum"} if QUEUE_TYPE == "quorum" else {}
```

Vì vậy `orders.main.q`, `orders.retry.q` và `orders.dlx` đều được tạo dưới dạng quorum queue.

5.2 Khai báo main queue

```
channel.exchange_declare(MAIN_EXCHANGE, "direct", durable=True)
```

```
channel.queue_declare(
    MAIN_QUEUE,
    durable=True,
    arguments={
        **qargs,
        "x-dead-letter-exchange": DLX,
        "x-dead-letter-routing-key": "dead",
    },
)
```

```
channel.queue_bind(MAIN_QUEUE, MAIN_EXCHANGE, "order")
```

`durable=True` nghĩa là định nghĩa exchange và queue tồn tại sau khi RabbitMQ restart.

Main queue có cấu hình dead-letter tới `orders.dlx`. Tuy nhiên trong luồng lỗi đang được test, consumer tự publish sang retry hoặc DLQ rồi ACK message cũ. Consumer không gọi `basic_nack(requeue=False)`, nên cấu hình DLX trên main queue không phải cơ chế chính tạo ra luồng retry này.

5.3 Khai báo retry queue

```
channel.exchange_declare(RETRY_EXCHANGE, "direct", durable=True)

channel.queue_declare(
    RETRY_QUEUE,
    durable=True,
    arguments={
        **qargs,
        "x-message-ttl": 3000,
        "x-dead-letter-exchange": MAIN_EXCHANGE,
        "x-dead-letter-routing-key": "order",
    },
)

channel.queue_bind(RETRY_QUEUE, RETRY_EXCHANGE, "retry")
```

Retry queue có ba cấu hình quan trọng:

Cấu hình	Giá trị	Ý nghĩa
x-message-ttl	3000 ms	Message chờ khoảng 3 giây
x-dead-letter-exchange	orders.main	Khi hết TTL, gửi message tới main exchange
x-dead-letter-routing-key	order	Main exchange route message tới orders.main.q

Đây là điểm RabbitMQ tự động thực hiện việc trì hoãn. Consumer không ngủ 3 giây để retry. Consumer publish message vào retry queue rồi tiếp tục làm việc khác.

5.4 Khai báo DLQ

```
channel.exchange_declare(DLX, "direct", durable=True)
channel.queue_declare(DLQ, durable=True, arguments=qargs)
channel.queue_bind(DLQ, DLX, "dead")
```

Binding cuối cùng:

```
orders.dlx -- key "dead" --> orders.dlq
```

orders.dlq không có consumer trong lab. Vì vậy message vào đây sẽ nằm ở trạng thái messages_ready để người vận hành kiểm tra.

6. Consumer Nhận Message Như Thế Nào?

Consumer đăng ký với main queue:

```
ch.basic_qos(prefetch_count=10)
ch.basic_consume(MAIN_QUEUE, handle, auto_ack=False)
ch.start_consuming()
```

Ý nghĩa:

- Consumer chỉ consume từ orders.main.q.
- Consumer không consume trực tiếp từ orders.retry.q.
- Consumer không consume từ orders.dlq.
- prefetch_count=10 giới hạn tối đa 10 message đã giao cho consumer nhưng chưa ACK.

- `auto_ack=False` yêu cầu code chủ động gọi ACK.

Khi RabbitMQ giao message cho consumer:

```
orders.main.q.messages_ready giảm
orders.main.q.messages_unacknowledged tăng
```

Message chưa bị xóa khỏi RabbitMQ. RabbitMQ đợi consumer ACK.

7. Consumer Xử Lý Poison Message

Hàm xử lý bắt đầu bằng việc đọc body và header:

```
data = json.loads(body)
retry_count = int((props.headers or {}).get("retry_count", 0))
```

Consumer cập nhật trạng thái Redis thành processing, rồi giả lập thời gian xử lý:

```
redis_client().setex(
    f"job:{data['id']}",
    86400,
    json.dumps({"**data", "status": "processing"}),
)

time.sleep(0.25)
```

Sau đó code phát hiện poison message:

```
if data.get("simulate_error"):
    raise RuntimeError("simulated poison message")
```

Vì `simulate_error=true`, code thành công phía dưới không bao giờ chạy:

```
es_client().index(...)
redis_client().setex(... status="processed")
ch.basic_ack(...)
```

Thay vào đó, execution chuyển xuống except.

8. Logic Quyết Định Retry Hay DLQ

Cấu hình consumer:

```
MAX_RETRIES: "3"
RETRY_DELAY_MS: "3000"
```

Trong code:

```
if retry_count < MAX_RETRIES:
    publish(ch, RETRY_EXCHANGE, "retry", body, props, retry_count + 1)
    outcome = "order_retry_scheduled"
else:
    publish(ch, DLX, "dead", body, props, retry_count)
    outcome = "order_dead_lettered"

ch.basic_ack(method.delivery_tag)
```

Câu hỏi quyết định là:

`retry_count` hiện tại có nhỏ hơn 3 không?

- Nếu có: tạo bản message tiếp theo trong retry queue.

- Nếu không: đưa message vào DLQ.
- Sau khi publish bản tiếp theo, ACK bản cũ trong main queue.

Hàm publish() giữ nguyên body, sao chép headers và thay retry_count:

```
headers = dict(props.headers or {})
headers["retry_count"] = retry_count

ch.basic_publish(
    exchange,
    key,
    body,
    properties=BasicProperties(
        delivery_mode=2,
        content_type="application/json",
        correlation_id=props.correlation_id,
        headers=headers,
    ),
)
```

9. Hành Trình Chi Tiết Của Một Message

9.1 Lần xử lý đầu tiên

API publish:

```
exchange = orders.main
routing key = order
retry_count = 0
```

RabbitMQ route message vào orders.main.q. Consumer nhận message và thấy:

```
simulate_error = true
retry_count = 0
```

Consumer phát sinh lỗi. Điều kiện:

$0 < 3$ = đúng

Consumer publish một bản sang:

```
exchange = orders.retry
routing key = retry
retry_count = 1
```

RabbitMQ route bản mới vào orders.retry.q. Sau đó consumer ACK bản cũ có retry_count=0, nên bản cũ được xóa khỏi orders.main.q.

9.2 Retry lần 1

Message retry_count=1 nằm trong orders.retry.q khoảng 3 giây.

Khi TTL hết hạn, RabbitMQ tự dead-letter message:

```
orders.retry.q
--> orders.main exchange
--> routing key order
--> orders.main.q
```

Consumer nhận message và lại phát sinh lỗi. Điều kiện:

$1 < 3 = \text{đúng}$

Consumer publish bản mới vào retry queue với:

```
retry_count = 2
```

Sau đó ACK bản cũ có `retry_count=1`.

9.3 Retry lần 2

Sau khoảng 3 giây nữa, RabbitMQ chuyển message `retry_count=2` từ retry queue về main queue.

Consumer xử lý lỗi. Điều kiện:

$2 < 3 = \text{đúng}$

Consumer publish bản mới vào retry queue với:

```
retry_count = 3
```

Sau đó ACK bản cũ có `retry_count=2`.

9.4 Retry lần 3 và chuyển DLQ

Sau khoảng 3 giây nữa, message `retry_count=3` quay về main queue.

Consumer xử lý lỗi. Điều kiện:

$3 < 3 = \text{sai}$

Consumer không publish vào retry exchange nữa. Nó publish vào:

```
exchange = orders.dlx  
routing key = dead  
retry_count = 3
```

RabbitMQ route message vào `orders.dlq`. Consumer cập nhật Redis:

```
status = dead_lettered  
error = simulated poison message
```

Cuối cùng consumer ACK bản trong `orders.main.q`.

10. Bảng Thời Gian Dễ Theo Dõi

Thời gian dưới đây là gần đúng. Mỗi lần xử lý còn mất khoảng 0,25 giây và có thêm thời gian giao nhận.

Thời điểm gần đúng	retry_count	Nơi message xuất hiện	Hành động
0 giây	0	orders.main.q	Consumer xử lý lỗi, lên lịch retry 1
0 đến 3 giây	1	orders.retry.q	Chờ TTL
khoảng 3 giây	1	orders.main.q	Consumer xử lý lỗi, lên lịch retry 2
3 đến 6 giây	2	orders.retry.q	Chờ TTL

Thời điểm gần đúng	retry_count	Nơi message xuất hiện	Hành động
khoảng 6 giây	2	orders.main.q	Consumer xử lý lỗi, lên lịch retry 3
6 đến 9 giây	3	orders.retry.q	Chờ TTL
khoảng 9 giây	3	orders.main.q	Consumer xử lý lỗi, chuyển DLQ
sau khoảng 9 giây	3	orders.dlq	Chờ người vận hành xử lý

MAX_RETRIES=3 nghĩa là có ba lần retry sau lần xử lý ban đầu. Vì vậy consumer xử lý message tổng cộng bốn lần.

11. Vì Sao Phải Publish Bản Mới Rồi ACK Bản Cũ?

Khi consumer nhận message từ main queue, RabbitMQ đang giữ message đó ở trạng thái unacknowledged.

Nếu consumer chỉ ACK ngay:

```
ACK bản cũ
không tạo bản mới
```

thì message biến mất và không thể retry.

Nếu consumer chỉ publish bản mới nhưng không ACK bản cũ:

```
bản mới nằm trong retry queue
bản cũ vẫn chưa hoàn tất trong main queue
```

thì khi connection consumer mất, RabbitMQ có thể đưa bản cũ về ready, gây xử lý trùng.

Do đó code dùng thứ tự:

1. Publish bản tiếp theo vào retry queue hoặc DLQ
2. ACK bản hiện tại trong main queue

Ý tưởng là chỉ xóa bản hiện tại sau khi đã tạo đường đi tiếp theo.

Tuy nhiên code consumer hiện chưa bật publisher confirm cho các lần republish. Trong hệ thống production, cần bảo đảm publish bản mới thực sự thành công trước khi ACK bản cũ; nếu không, vẫn có cửa sổ mất message.

12. Hiểu Log Consumer

Mỗi lần còn được retry, consumer ghi:

```
message = order_retry_scheduled
retry_count = 0
```

Sau đó lần lượt là:

```
order_retry_scheduled, retry_count = 1
order_retry_scheduled, retry_count = 2
```

Ở lần cuối:

```
order_dead_lettered, retry_count = 3
```

Lưu ý: log ghi retry_count của message vừa thất bại, không phải count của bản mới vừa publish.

Vì vậy chuỗi log mong đợi là:

```
retry_count=0 -> scheduled
retry_count=1 -> scheduled
retry_count=2 -> scheduled
retry_count=3 -> dead_lettered
```

13. Hiểu Kết Quả list_queues

Lệnh theo dõi:

```
watch -n 1 "docker compose -f docker-compose.full.yml exec -T rabbitmq1 \
  rabbitmqctl list_queues name messages_ready messages_unacknowledged"
```

Hai metric quan trọng:

- messages_ready: message đang nằm trong queue và chưa được giao cho consumer.
- messages_unacknowledged: message đã giao cho consumer nhưng consumer chưa ACK.

Trong lúc test, message di chuyển khá nhanh. watch -n 1 có thể bỏ lỡ trạng thái ngắn.

Ví dụ khi message đang chờ retry:

```
orders.main.q ready=0 unacknowledged=0
orders.retry.q ready=1 unacknowledged=0
orders.dlq ready=0 unacknowledged=0
```

Khi consumer đang xử lý message từ main:

```
orders.main.q ready=0 unacknowledged=1
```

Kết quả cuối cùng:

```
orders.main.q ready=0 unacknowledged=0
orders.retry.q ready=0 unacknowledged=0
orders.dlq ready=1 unacknowledged=0
```

orders.dlq.messages_ready=1 nghĩa là poison message đã hoàn thành toàn bộ chu trình retry và đang chờ điều tra.

14. Phần Nào Do Code, Phần Nào Do RabbitMQ?

Hành động	Thành phần thực hiện
Tạo poison request	Client test
Tạo message và đặt retry_count=0	API
Route message từ exchange tới queue	RabbitMQ
Phát hiện simulate_error=true	Consumer
Quyết định còn retry hay hết retry	Consumer
Tăng retry_count	Consumer
Publish message vào retry exchange	Consumer
Giữ message trong 3 giây	RabbitMQ retry queue

Hành động	Thành phần thực hiện
Khi hết TTL, đưa message về main exchange	RabbitMQ
Publish message cuối cùng vào DLX	Consumer
Route message vào orders.dlq	RabbitMQ
Giữ message trong DLQ	RabbitMQ

Điểm quan trọng nhất: RabbitMQ không tự biết lỗi nào cần retry ba lần. Consumer tự quyết định số lần retry bằng header `retry_count`. RabbitMQ chỉ thực hiện routing, lưu queue và trì hoãn bằng TTL theo cấu hình.

15. Tóm Tắt Ngắn Gọn

1. Client gọi API với `simulate_error=true`.
2. API publish message vào `orders.main`, đặt `retry_count=0`.
3. RabbitMQ route message vào `orders.main.q`.
4. Consumer nhận message, cố tình phát sinh lỗi.
5. Khi `retry_count < 3`, consumer tăng count và publish sang `orders.retry`.
6. RabbitMQ giữ message trong `orders.retry.q` khoảng 3 giây.
7. Hết TTL, RabbitMQ tự đưa message về `orders.main.q`.
8. Chu trình lặp lại cho các count 1, 2 và 3.
9. Khi nhận message có `retry_count=3`, consumer publish sang `orders.dlx`.
10. RabbitMQ route message vào `orders.dlq`, nơi message chờ người vận hành xử lý.

Các file code chính:

```
client/load_test.py
api-server/app/main.py
shared/common.py
consumer-worker/app/main.py
docker-compose.full.yml
```